DRAFT FOR REVIEW · OCTOBER 2026

Trading Crypto Futures with AI Agents

TradingView MCP, Binance MCP and Your Own Bot

Preface and detailed outline · written for Subodh, Delhi

Education only, not financial advice. Leveraged crypto trading can lose more money than you expect, and most retail traders who use leverage lose money.

Preface

This book teaches you to trade crypto futures with AI tools in a careful, testable way. You will use Claude to read charts through TradingView, to place and manage orders on Binance through the official Binance MCP server, and finally to run your own small trading bot. At every step the goal is the same: understand the mechanism, measure the risk, test on fake money or the smallest size, and only then use real capital.

It is written for one reader. You are a third-year BMS (finance) student in Delhi. You work as a mutual fund distributor, you are aiming for the CFA programme and a career in fund management, and you already write Python, Pine Script, SQL and some Rust. You have Rs 1,00,000 of trading capital and you trade crypto futures on Binance. Every example in the book is sized for that account.

What you will build

By the end of the book you will have a complete, working system. Figure P.1 shows all of it on one page, and each chapter builds one part.

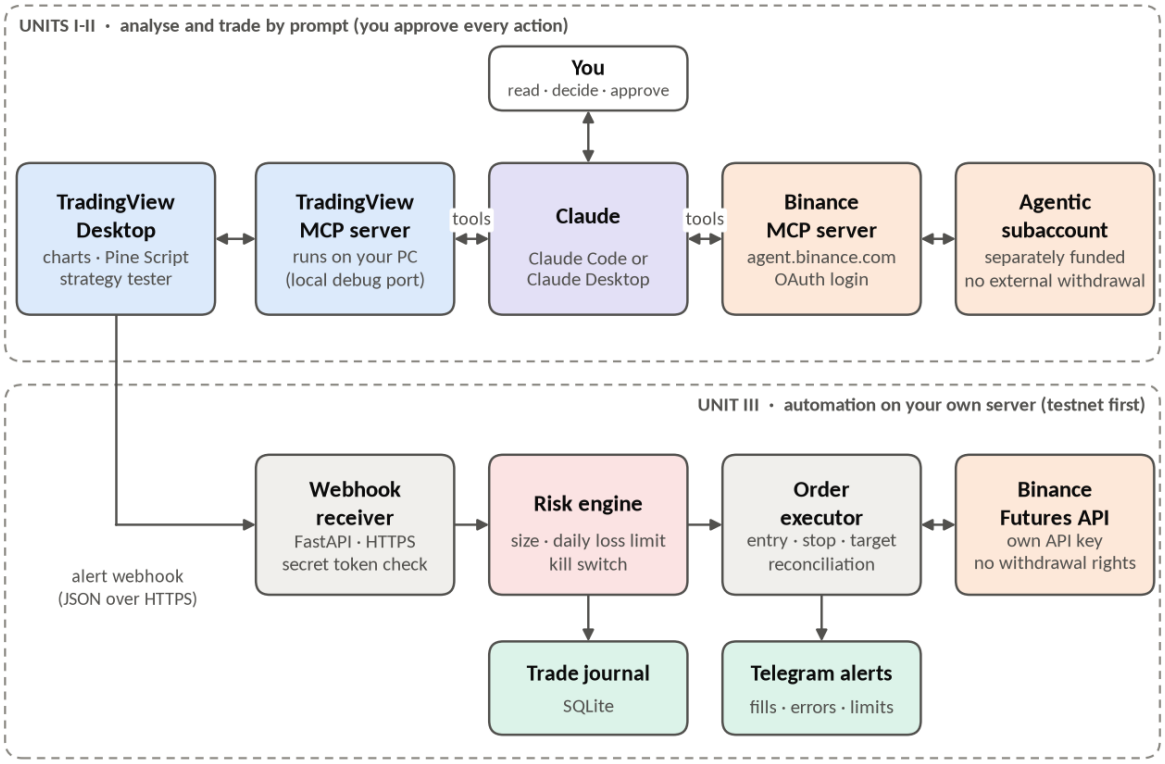


Figure P.1. The complete system. Top: you analyse with TradingView through Claude and trade by prompt through the Binance MCP server, approving every action. Bottom: the automated path from a TradingView alert to a Binance order, guarded by a risk engine.

The top half of Figure P.1 is **trading by prompt**. A **prompt** is the instruction you type to an AI model. Claude talks to two **MCP servers**. MCP (Model Context Protocol) is an open standard that lets an AI app connect to outside tools in one common way; an MCP server is the program that offers those tools. One server controls TradingView Desktop on your own computer, so Claude can read prices and indicators, draw levels and run Pine Script. The other is Binance's own MCP server, so Claude can read your balances and place orders, but only inside a separately funded **Agentic**

subaccount: a Binance subaccount you fund yourself and assign to the AI, which (according to Binance's launch material) cannot withdraw money to outside addresses.

The bottom half is **automation**. A Pine Script strategy on TradingView sends an alert as a **webhook** (an HTTP message sent to your web address when an event happens). Your own Python server receives it, checks a secret token, sizes the trade, passes it through a risk engine, and only then sends orders to the Binance Futures API. A journal database records everything and Telegram tells you what happened.

How the book is organised

The book has 4 units and 12 chapters. Figure P.2 shows the order in which they build on each other.

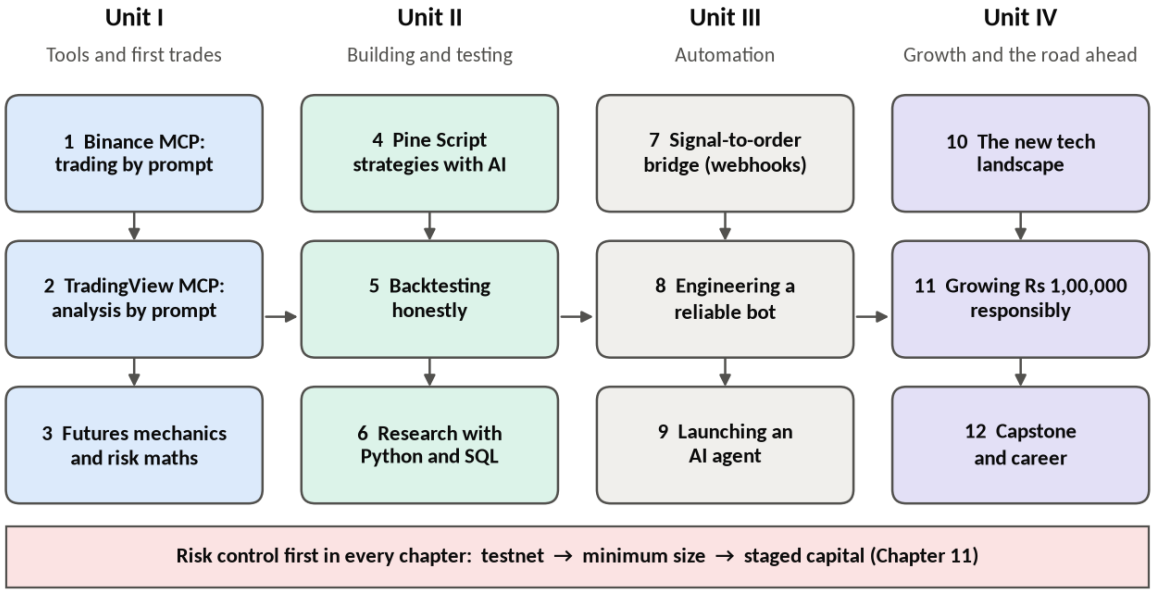


Figure P.2. The roadmap: four units, twelve chapters. Risk control runs underneath all of them.

- **Unit I, The tools and your first trades (Chapters 1-3).** You connect Claude to Binance and to TradingView, and you learn the mechanics and maths of perpetual futures: funding, leverage, margin modes, liquidation and position sizing.
- **Unit II, Building and testing strategies (Chapters 4-6).** You write three Pine Script strategies with Claude's help, learn to backtest honestly, and build a research toolkit in Python and SQL.
- **Unit III, Automation (Chapters 7-9).** You build the bridge from a TradingView alert to a Binance order, engineer it to be reliable, and then add AI agents for research and monitoring, with a human approving every trade.
- **Unit IV, Growth and the road ahead (Chapters 10-12).** You separate real edge from hype in new tools, plan how to grow capital in stages, handle Indian tax and records, and finish with a capstone review and a career portfolio.

How each chapter works

Every chapter follows the same pattern, so you always know where to look:

1. **Learning goals**: 3 to 5 things you will be able to do.
2. **Concepts from the basics**, with worked examples in rupees and USDT for a Rs 1,00,000 account.

3. **Copy-paste prompts** in blue boxes. After each one: what the tool does, and what a good response looks like.
4. **Complete code** (Python, Pine Script v6, SQL) included from files that were run or compile-checked. Anything that could not be tested is marked UNTESTED.
5. **At least four figures**: graphs from real or clearly labelled simulated data, and diagrams.
6. **Common mistakes**, a **hands-on exercise** with a checklist, and a **5-question self-test** with answers.

The honest maths of leveraged trading

Before any tool, one fact: **most retail traders who use leverage lose money.** The best large study of retail derivatives traders in India is SEBI's study of individual traders in equity futures and options. For the financial year 2024-25 it reported that more than 91% of individual traders made net losses, with an average loss of about Rs 1.1 lakh per trader. That study covers Indian equity derivatives, not crypto, but crypto futures have the same features that hurt retail traders (leverage, fees, fast moves) and run 24 hours a day.

> **NOT VERIFIED: SEBI figures**
> The SEBI numbers above are as reported by Business Standard (7 July 2025) and Value Research. The sebi.gov.in site could not be opened from the writing environment. Check the original study on sebi.gov.in under Reports and Statistics before quoting it.

Two simple pieces of maths explain much of why. The first is that losses hurt more than gains help. If Rs 1,00,000 falls to Rs 50,000 (a 50% loss), you need a 100% gain just to get back to where you started. Figure P.3 shows this curve.

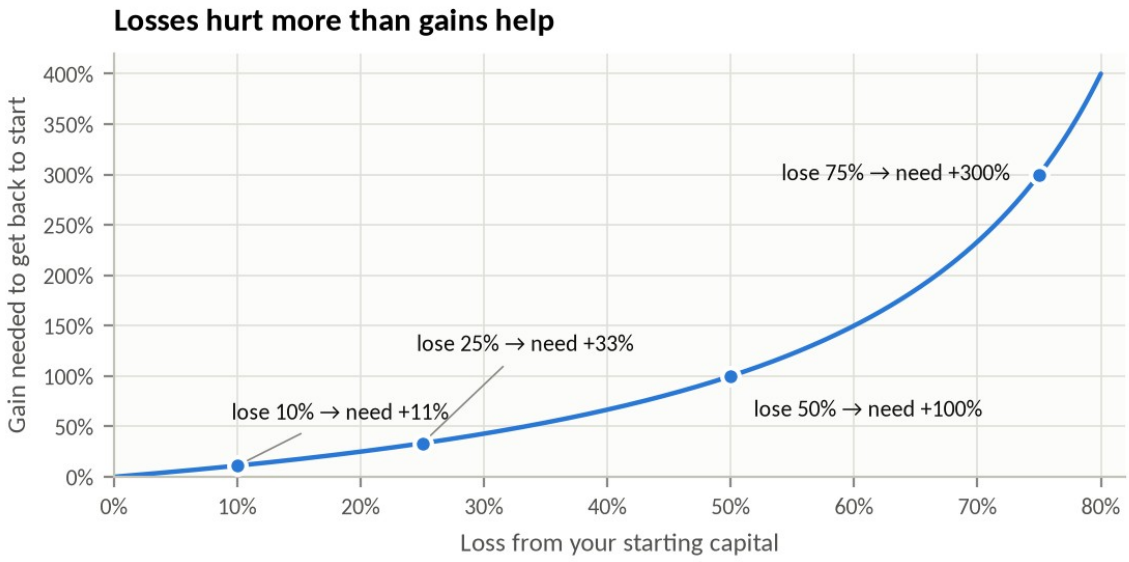
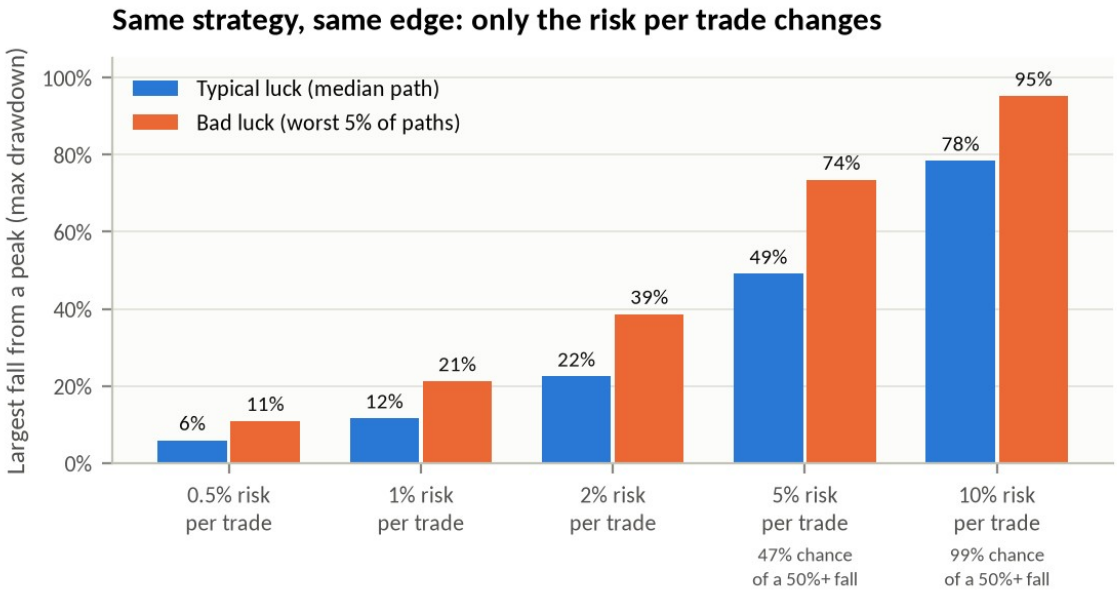


Figure P.3. The gain needed to recover from a loss grows faster than the loss itself.

The second is that the amount you risk on each trade matters more than almost anything else. Figure P.4 takes one simulated strategy that genuinely has an edge: it wins 45% of trades, wins 1.5 times what it risks, and so makes +0.125R per trade on average (**R** is the amount you risk on one trade; risking Rs 1,000 and making Rs 1,500 is +1.5R). Only the risk per trade changes.



SIMULATED DATA: 10,000 paths x 200 trades; win rate 45%, wins +1.5R, losses -1R (expectancy +0.125R per trade). Not market prices.

Figure P.4. The same winning strategy, simulated 10,000 times over 200 trades. At 1% risk per trade the typical worst fall is about 12%; at 10% risk it is about 78%, and a 50% fall is almost certain.

Read Figure P.4 slowly. At 1% risk per trade (Rs 1,000 on a Rs 1,00,000 account), the typical largest fall from a peak is about 12%, and in the unlucky 5% of paths about 21%. At 5% risk the typical fall is about half the account, and in nearly half the paths the account falls 50% or more at some point. At 10% risk a 50% fall happens in about 99% of paths. The strategy did not change; only the bet size did. Real strategies are usually weaker than this one, and real markets have fat tails (rare, very large moves) that this simple simulation leaves out.

That is why this book puts risk control ahead of returns in every chapter, and why it never promises profits, "financial freedom" or fast growth. Where returns are discussed (Chapter 11), you will see realistic compounding maths together with the drawdowns that come with it.

Safety rules used in this book

These rules apply in every chapter. They are not optional.

1. **Fake money or minimum size first.** Every exercise starts on the Binance Futures testnet (Binance's practice environment with fake funds) or at the smallest order size the exchange allows.
2. **The AI never trades without you.** Prompts that could change your account ask Claude to show a summary and wait for your explicit confirmation. You approve each order yourself.
3. **Separate, small pots of money.** The Agentic subaccount and the bot's subaccount hold only what you can afford to lose completely. Your main account is never connected to automation.
4. **No secrets in files.** API keys live in environment variables. The project ships an `.env.example` file with empty values; your real `.env` is never committed or shared.
5. **No withdrawal permission, ever.** API keys used by bots are created without withdrawal rights and, where possible, restricted to your server's IP address.
6. **Limits outside the AI.** Daily loss limits, maximum position size and a kill switch live in code and in account funding, not in a prompt the model might misread.

7. **The author never trades for you.** While writing this book, Claude does not place, modify or cancel any real order. All examples are illustrative or run on testnet.

Conventions

Table P.1 lists the conventions used throughout the book.

Table P.1. Conventions used in this book.

Item	Convention
Rupees	Indian grouping: Rs 1,00,000; Rs 12,500
USDT	1,000.00 USDT; Binance futures are priced in USDT
Exchange rate	**1 USDT = Rs 100** for easy maths. App quotes on 4-5 October 2026 were about Rs 96 to Rs 101. Always check the live rate.
Prices in examples	Labelled "illustrative" unless they come from a dated data source
Times	UTC for exchange data, IST for your daily routine; always labelled
Default risk	1% of equity per trade (Rs 1,000 = 10 USDT) in worked examples
Code	Python 3.11+, Pine Script v6, SQLite. Files live in `code/chXX/`
Figures	Numbered per chapter (Figure 3.2); every chart says whether data is real or SIMULATED

You will meet five kinds of boxes. A blue **prompt box** holds text to copy into Claude. A grey **code box** holds a program. Coloured callouts mark a **Risk**, a **Note**, a **Tip**, a **Checklist**, or a fact that is **Not verified**. Here is what a prompt box looks like:

> **PROMPT · Binance MCP (read-only)**
> List the MCP tools you can see from the Binance server, grouped by area (market data, account, spot, futures, margin, convert). For each tool give its exact name and say whether it only reads data or can change my account. Do not call any tool that places, changes or cancels an order.

What the tool does. Nothing is called on Binance; Claude reads the tool list its MCP client already received when it connected. **A good response looks like** a grouped table of exact tool names with a clear "read" or "write" label. A red flag is any tool name Claude cannot show you from the actual list, which means it is guessing.

How facts were checked

Crypto tools change monthly. Every chapter is researched just before it is written, using official sources first: the Binance developer documentation and announcements, the TradingView Pine Script reference and Help Center, the Income Tax Department and CBDT, and Anthropic's documentation for Claude. Each chapter ends with a "Sources and verification" list with the date each source was checked.

When something could not be verified against an official source, the text says so in a **Not verified** callout and tells you where to check. The project file `PROGRESS.md` keeps a running list of these items. For this preface and outline, several official sites (binance.com, developers.binance.com, tradingview.com and the income tax portal) could not be opened from the writing environment, so facts about them were taken from search results and public GitHub repositories and are marked for re-checking.

The tool names in the prompts come from your own installed MCP servers. Before Chapter 1 is written, you will paste the exact tool lists from your TradingView MCP and Binance MCP servers, and every prompt will be written against those names.

What you need before Chapter 1

- ☐ A Binance account with KYC done and two-factor authentication (2FA) switched on.
- ☐ Access to Binance Agent OS and an Agentic subaccount (check availability for Indian users in the Binance app).
- ☐ Claude Code or Claude Desktop, signed in, able to add MCP servers.
- ☐ TradingView Desktop and a TradingView MCP server installed and connected to Claude.
- ☐ Python 3.11 or newer, Node.js, Git, and a code editor such as VS Code.
- ☐ A Binance Futures testnet (demo trading) account for the practice exercises.
- ☐ A TradingView Essential plan (or higher) by Unit III, for webhook alerts.

> **RISK: Education, not financial advice**
> This book is for education only. It is not financial, investment, legal or tax advice, and nothing in it is a recommendation to buy or sell any asset. Crypto futures are high-risk, leveraged products; you can lose all the money you put in. Indian tax treatment of crypto futures is not fully settled; consult a chartered accountant for your own situation. You are responsible for your own trades.

Detailed Outline

This outline is the plan for all 12 chapters. Please read it, change anything you like, and approve it before Chapter 1 is written. Each chapter will follow the fixed template from the Preface (learning goals, concepts with worked examples, boxed prompts, tested code, at least four figures, common mistakes, a hands-on exercise with a checklist, a 5-question self-test, and a "Sources and verification" list). So that this outline stays readable, those fixed parts are only summarised below.

Table 0.1 gives the size of each chapter at a glance.

Table O.1. Chapter plan at a glance.

Ch	Title	Target pages	Figures planned	Main code	You finish with
1	Binance MCP: trading by prompt	18-20	6	order preview calculator (Python)	Claude connected to your Agentic subaccount; one safe order placed and cancelled
2	TradingView MCP: analysing the market by prompt	16-18	5	indicator checker (Python)	a daily analysis routine you run in 10 minutes
3	Futures mechanics and risk maths	18-20	6	position-size calculator with tests	a calculator you trust, checked against Binance
4	Pine Script strategies with AI	18-20	6	3 Pine v6 strategies + Python replicas	three strategies compiled and backtested with costs
5	Backtesting honestly	17-19	6	backtester, walk-forward, scorecard	a pass/fail verdict for each strategy
6	Research with Python and SQL	17-19	6	data fetchers, SQLite journal, stats, Monte Carlo	a trade journal database and review queries
7	The bridge: TradingView signal to Binance order	17-19	5	FastAPI webhook receiver + tests	testnet orders placed from a TradingView alert
8	Engineering a reliable bot	19-20	6	bot package (REST, WebSocket, risk, reconcile, Telegram)	a bot running on a VPS against testnet
9	Launching an AI agent	17-19	5	research and monitoring agents (Agent SDK)	agents that report and alert, with human approval for trades
10	The new technology landscape	15-17	5	two "test the claim" studies	a checklist to judge any new tool
11	Growing Rs 1,00,000 responsibly	17-19	5	capital-ladder simulator, VDA tax estimator	a written trading plan and record-keeping system
12	Capstone and career	15-17	5	live-vs-backtest report	a 90-day review and a career portfolio

Total: about 210 to 230 pages, plus front matter, glossary and an index of prompts.

Unit I · The tools and your first trades

Chapter 1. Binance MCP: trading by prompt

Why it matters. This is the first time an AI will touch your money. The chapter makes sure you understand exactly what Claude can and cannot do on your account before it does anything.

Learning goals

- Explain MCP in plain words: client, server, tools, and how one tool call happens.
- Connect Claude to the official Binance MCP server, link the Agentic subaccount, set permissions, and revoke access.
- Use read-only prompts for market data, balances, positions and order history.
- Place, check, cancel and close every common futures order type by prompt, at minimum size, after a safety checklist.

Sections

- **1.1 What MCP is**, from the basics: host, client, server; tools, resources and prompts; local (stdio) vs remote (HTTP) servers; the lifecycle of one tool call; why the model never sees your password.
- **1.2 Binance Agent OS in one page**: its parts (Binance APIs, Wallet Agentic Hub, x402 payments, Skill Hub, MCP server) and which chapter uses each.
- **1.3 The Agentic subaccount**: how you fund it, what the agent can do (reported: view balances and positions; trade spot, margin, convert, USDS-M and COIN-M futures; move funds between wallets inside the subaccount) and cannot do (withdraw externally; pull money from your main account). How much to put in during the learning phase (example: Rs 10,000 = 100 USDT).
- **1.4 Setup with Claude**, step by step: add the remote server, OAuth login on Binance's own page, choose the subaccount, approve scopes, test with a read-only prompt, revoke access. Separate steps for Claude Code and Claude Desktop.
- **1.5 Permissions in two places**: Binance-side scopes and confirmations, and Claude-side tool approval. Rule: read tools may be pre-approved; write tools are never "always allowed".
- **1.6 Feature tour**, one prompt per feature: market data (price, 24h stats, order book, candles, funding, exchange rules), account (balances, positions, open orders, order and trade history, funding income), spot, futures, margin, convert. A table maps each feature to the exact tool name on your server.
- **1.7 Futures orders by prompt**: market; limit (GTC, IOC, FOK, post-only); stop-loss; take-profit; reduce-only; close-position; set leverage; set margin mode (isolated or cross); check position mode (one-way or hedge); modify; cancel one; cancel all; close a position.
- **1.8 One complete worked trade**: BTCUSDT long, equity 1,000 USDT (Rs 1,00,000), risk 1% = 10 USDT. Entry 60,000 and stop 59,400 (illustrative) give 10 / 600 = 0.0167 BTC, rounded down to the step size: 0.016 BTC = 960 USDT notional. At 3x isolated, margin is 320 USDT. Taker fees at an assumed 0.05% each way add about 0.96 USDT (Rs 96), so the true risk is about 10.56 USDT (Rs 1,056).
- **1.9 The pre-order safety checklist prompt**: Claude fetches live data, checks 12 items (symbol, side, quantity vs step size, minimum notional, price vs tick size, leverage, margin mode, position mode, stop present, reduce-only on exits, risk in Rs vs your limit, liquidation price beyond the stop), prints a table, and stops until you type CONFIRM.
- **1.10 Prompt cheat sheet**: one page with every prompt from the chapter.

Prompts (about 30). Tool discovery; read-only market and account prompts; one prompt per order type; the safety checklist; end-of-session clean-up ("list and cancel orphan orders"). All written against the exact tool names you paste from your server.

Code. `code/ch01/order_preview.py`: turns an order idea into the same summary table the checklist prompt prints (quantity rounding, notional, margin, risk in Rs and USDT), so you can check Claude's arithmetic offline. Tested with pytest.

Table O.2. Figures planned for Chapter 1.

Figure	Kind	Content
1.1	diagram	MCP architecture: Claude (host and client), MCP servers, tools
1.2	diagram	Money flow: main account, Agentic subaccount wallets, allowed and blocked moves
1.3	sequence	Connecting Claude to Binance with OAuth
1.4	sequence	One order tool call with two approvals (Claude, then Binance)
1.5	graph	Order types drawn on a price path: entry, stop, take-profit, reduce-only exit (SIMULATED path)
1.6	flowchart	The pre-order safety checklist

Common mistakes. Quantity in USDT vs in coins; missing reduce-only so an exit opens a new position; orphan stop or take-profit orders after a manual close; leverage changed but margin mode left on cross; "always allow" on write tools; trusting a price the model remembers instead of one it just fetched; funding the subaccount with more than the learning budget.

Hands-on exercise. Connect; list tools; run the read-only routine; place a minimum-size limit order far from the market (so it cannot fill), confirm it appears, cancel it; optional minimum-size futures round trip at 1x-2x isolated with a stop. Checklist of 10 items.

Verify before writing. Binance MCP documentation (developers.binance.com, Agent-native section): endpoint, OAuth steps, permission scopes, tool names, whether testnet is supported, whether each write action needs in-app confirmation, availability for Indian accounts. Also whether futures stop and take-profit orders now go through Binance's separate algo-order service (reported change in late 2025).

Chapter 2. TradingView MCP: analysing the market by prompt

Why it matters. Good orders need good analysis. Claude can read charts quickly, but it can also misread them. This chapter teaches you to get numbers from data tools, not guesses from pictures.

Learning goals

- Explain how a community TradingView MCP server controls TradingView Desktop on your computer.
- Control charts, pull OHLCV data and indicator values, and draw levels by prompt.
- Run a multi-timeframe analysis and a 10-minute daily routine that ends in a written plan, not a trade.
- Practise in replay mode and know the limits of AI chart reading.

Sections

- **2.1 How it works**: TradingView Desktop started with a local debugging port (Chrome DevTools Protocol), a Node.js MCP server on your PC, Claude as the client. Security (local only), Terms of Use cautions, and the fact that these servers are not made by TradingView.
- **2.2 Setup and health check**; symbols for Binance perpetuals on TradingView (for example `BINANCE:BTCUSDT.P`).
- **2.3 Chart control**: symbol, timeframe, chart type, visible range, layouts and panes.
- **2.4 Data extraction**: quotes, OHLCV with compact summaries (to save Claude's context), indicator values, and values drawn by Pine scripts (lines, labels, tables, boxes).
- **2.5 Indicators from the basics**: EMA, RSI, ATR, volume; adding them and reading values.
- **2.6 Multi-timeframe analysis**: 1D trend, 4H structure, 1H trigger.
- **2.7 Drawing levels and screenshots**: support and resistance from swing highs and lows, with prices listed as numbers.

- **2.8 Alerts and watchlists** (price alerts now; webhook alerts in Chapter 7).
- **2.9 Replay mode practice**: step bar by bar, place practice trades, review results.
- **2.10 The daily analysis routine prompt (IST times)**, producing a one-page plan table.
- **2.11 Full prompt library.**
- **2.12 Limits of AI chart reading**: screenshot misreads, invented levels, context limits on big data pulls, confirmation bias, and why analysis is not the same as edge.

Code. `code/ch02/indicators.py` (EMA, Wilder RSI, ATR from OHLCV) and `code/ch02/verify_readings.py`, which compares values read through the MCP with values computed in Python. Tested with pytest against known values.

Table O.3. Figures planned for Chapter 2.

Figure	Kind	Content
2.1	diagram	Claude, MCP server, debugging port, TradingView Desktop, TradingView servers
2.2	graph	BTCUSDT perpetual 4H with EMA 20/50/200, ATR panel below (real data if available)
2.3	diagram	Multi-timeframe funnel: 1D trend, 4H structure, 1H trigger
2.4	flowchart	The daily routine, with times in IST
2.5	graph	Values read through the MCP vs values recomputed in Python

Common mistakes. Reading numbers off screenshots; pulling 5,000 bars into the chat; mixing spot and perpetual symbols; letting the routine turn into a trade instruction; trusting levels the model drew without listing their prices.

Hands-on exercise. Run the daily routine on 5 days, save each plan, then compare the plans with what happened. Replay one past week bar by bar. Checklist of 8 items.

Verify before writing. The exact tool list and version of your TradingView MCP server; TradingView symbol naming for Binance perpetuals; alert limits per plan; TradingView Terms of Use.

Chapter 3. Futures mechanics and risk maths

Why it matters. Most blown-up accounts come from not understanding leverage, margin and liquidation. After this chapter you can size any trade so that a stop-loss costs exactly what you planned.

Learning goals

- Explain perpetual futures, last price, index price, mark price and funding.
- Compare isolated and cross margin, and compute a liquidation price.
- Size a position from risk: quantity = (equity x risk %) / |entry - stop|, then round to the exchange's rules.
- Include fees, slippage and funding in the true cost of a trade.
- Use and test a Python position-size calculator.

Sections

- **3.1 Spot vs futures**; what a perpetual contract is; USDS-M vs COIN-M; contract rules (tick size, step size, minimum notional).
- **3.2 Three prices**: last, index and mark. Why liquidation uses the mark price.
- **3.3 Funding**: the formula, intervals, who pays whom. Worked example: 960 USDT long at +0.01% per 8 hours costs about 0.096 USDT (Rs 9.6) per payment, about Rs 29 a day and about Rs 860 over 30 days, almost one full Rs 1,000 risk unit.

- **3.4 Leverage**, initial margin, maintenance margin and margin ratio.
- **3.5 Isolated vs cross margin**, with the same trade shown both ways.
- **3.6 Liquidation price**: the simple approximation (isolated long at 10x from 60,000 is roughly 60,000 x (1 - 1/10 + maintenance rate)) and Binance's exact method with leverage brackets.
- **3.7 Position sizing from risk**, step by step, for a Rs 1,00,000 account; why leverage is an output of sizing, not an input.
- **3.8 Fees and slippage**: maker vs taker; round-trip cost; why tight stops make fees a large share of risk.
- **3.9 The Python position-size calculator**, its tests, and checking it against Binance's own calculator.
- **3.10 Rare but real**: insurance fund and auto-deleveraging.

Code. `code/ch03/position_size.py`, `liquidation.py`, `fees.py`, and `test_*.py` (pytest). The calculator reads symbol rules from a saved exchange-information file so it works offline.

Table O.4. Figures planned for Chapter 3.

Figure	Kind	Content
3.1	graph	Profit and loss vs price move: spot 1x vs futures 3x and 10x
3.2	diagram	How the index price, mark price and last price are formed and used
3.3	graph	BTCUSDT funding rate history and the cumulative cost of holding (real data if available)
3.4	diagram	Isolated vs cross margin: what can be lost
3.5	graph	Distance to liquidation vs leverage
3.6	graph	Fees as a share of risk vs stop distance

Common mistakes. Choosing leverage first and size second; ignoring funding on multi-day holds; using last price instead of mark price; stops placed beyond the liquidation price; cross margin with several open positions; forgetting that fees are charged on notional, not margin.

Hands-on exercise. Size five given trades by hand, then with the calculator, then check each on Binance's futures calculator (testnet). Checklist of 8 items.

Verify before writing. Binance USDS-M fee schedule for regular users, funding intervals, leverage and maintenance-margin brackets, minimum notional by symbol, Binance's liquidation price formula.

Unit II · Building and testing strategies

Chapter 4. Pine Script strategies with AI

Why it matters. A strategy is a set of rules you can test. Claude can write Pine Script quickly, but only a disciplined loop of describe, generate, compile and backtest turns that speed into reliable code.

Learning goals

- Turn an indicator into a strategy with entries, exits, stops and risk-based sizing in Pine v6.
- Run the describe-generate-compile-backtest loop through the TradingView MCP.
- Build and explain a trend, a mean-reversion and a breakout strategy.
- Set realistic strategy properties: capital, commission, slippage, order fill rules.

Sections

- **4.1 Pine v6 essentials**: script types, series and history, inputs, `strategy()` settings, `request.security()` without lookahead.
- **4.2 From indicator to strategy**: `strategy.entry`, `strategy.exit` with stop and limit, quantity from risk (ATR-based stop).
- **4.3 The loop through the MCP**: write a specification, generate, inject, compile, read errors, fix, backtest, read the Strategy Tester. Rule: Claude edits one thing at a time.
- **4.4 Strategy 1**, trend following: EMA cross with a trend filter and ATR stop.
- **4.5 Strategy 2**, mean reversion: RSI and Bollinger Bands in ranging conditions, with a trend filter that stands aside in strong trends.
- **4.6 Strategy 3**, breakout: Donchian channel with a volume filter and ATR stop.
- **4.7 Alerts from strategies**: `alert()` messages in JSON (used in Chapter 7).

Code. `code/ch04/trend_ema_atr.pine`, `mean_reversion_rsi_bb.pine`, `breakout_donchian.pine`, and `code/ch04/replica.py` (the same logic in Python, used for figures and cross-checking). Pine compile status: compiled through your TradingView MCP. The writing environment cannot reach TradingView, so each Pine file stays marked "compile pending" until you run the compile prompt and paste the result.

Table O.5. Figures planned for Chapter 4.

Figure	Kind	Content
4.1	diagram	The describe-generate-compile-backtest loop through the MCP
4.2	diagram	Anatomy of a Pine v6 strategy script
4.3	graph	Trend strategy entries and exits on price
4.4	graph	Mean-reversion strategy entries and exits on price
4.5	graph	Breakout strategy with Donchian channel
4.6	graph	Equity curves of the three strategies after costs (Python replica)

Common mistakes. Default strategy settings with zero commission; percent-of-equity sizing that ignores the stop; `request.security()` with lookahead; letting Claude rewrite the whole script to fix one error; judging a strategy on one symbol and one timeframe.

Hands-on exercise. Build one of the three strategies through the loop on BTCUSDT 1H, record the Strategy Tester summary, and match it with the Python replica. Checklist of 9 items.

Verify before writing. Pine Script v6 reference for every function used; Strategy Tester settings and fill assumptions; which MCP tool on your server returns Strategy Tester results.

Chapter 5. Backtesting honestly

Why it matters. It is easy to make any idea look profitable on past data. This chapter is about catching the ways a backtest lies before real money finds out.

Learning goals

- Read every number in the Strategy Tester and know what it hides.
- Detect repainting, lookahead bias and overfitting.
- Apply realistic fees, slippage and funding.
- Run in-sample/out-of-sample and walk-forward tests, split results by market regime, and give each strategy a pass or fail.

Sections

- **5.1 Reading the Strategy Tester**: net profit, max drawdown, profit factor, win rate, average trade, trade list, and what each one can hide.
- **5.2 Repainting**: where it comes from in Pine and how to test for it.
- **5.3 Lookahead bias**, in Pine and in Python (signal on a bar's close, filled at that same close).
- **5.4 Overfitting**: parameter grids, plateaus vs spikes, the number-of-trials problem.
- **5.5 Realistic costs**: taker fees, slippage in ticks, funding over the holding period, and a 2x-cost stress test.
- **5.6 In-sample vs out-of-sample**, and walk-forward analysis.
- **5.7 Market regimes**: trend, range, high and low volatility; results by regime.
- **5.8 The pass/fail scorecard (draft criteria**: at least 100 in-sample and 30 out-of-sample trades; out-of-sample profit factor of at least 1.2 after costs; still positive at 2x costs; max drawdown at 1% risk within 20%; nearby parameters also profitable; positive in at least two of three regimes; no repainting; Python and Pine results agree).

Code. `code/ch05/backtest.py` (next-bar execution, costs), `walk_forward.py`, `regimes.py`, `scorecard.py`, with tests.

Table O.6. Figures planned for Chapter 5.

Figure	Kind	Content
5.1	diagram	Repainting: the signal seen live vs the signal shown later
5.2	graph	Equity with lookahead bias vs without
5.3	graph	Parameter heatmap: a fragile spike vs a robust plateau
5.4	graph	Equity before and after fees, slippage and funding
5.5	diagram	Walk-forward windows
5.6	graph	Results by market regime

Common mistakes. Optimising on all the data; reporting the best of 500 runs; ignoring the number of trades; using close-to-close drawdown; forgetting funding; changing rules after seeing out-of-sample results.

Hands-on exercise. Put all three Chapter 4 strategies through the scorecard and write a one page verdict for each. Checklist of 8 items.

Verify before writing. TradingView Help Center definitions of Strategy Tester metrics and fill assumptions (including Bar Magnifier and plan availability).

Chapter 6. Research with Python and SQL

Why it matters. A fund analyst works from data, not from screenshots. This chapter gives you your own research desk: data pipelines, a trade journal in SQL, and the statistics used to judge a strategy.

Learning goals

- Pull candles, funding and open interest into pandas, with times handled in UTC and IST.
- Load TradingView Strategy Tester trade lists into pandas.
- Build a SQLite trade-journal database and write review queries.
- Compute Sharpe, Sortino, drawdown and expectancy, and run a Monte Carlo simulation.

Sections

- **6.1 Getting candles**: Binance public endpoints, bulk historical files, and exports through the TradingView MCP; time zones.
- **6.2 Loading Strategy Tester trade lists.**

- **6.3 The trade journal in SQL**: schema (trades, orders, fills, funding, daily equity, setups, mistakes), inserts, and review queries (win rate by setup, by IST hour, by weekday; R distribution; the cost of each mistake type).
- **6.4 Funding and open interest data**, and what they can and cannot tell you.
- **6.5 Performance statistics**: returns, Sharpe and Sortino (annualised for 365-day markets), max drawdown, Calmar, expectancy.
- **6.6 Monte Carlo**: reshuffling and resampling trades to see the range of drawdowns.

Code. `fetch_klines.py`, `fetch_funding_oi.py`, `load_tv_trades.py`, `journal_schema.sql`, `journal.py`, `review_queries.sql`, `stats.py`, `monte_carlo.py`, with pytest tests.

Table O.7. Figures planned for Chapter 6.

Figure	Kind	Content
6.1	diagram	Research data pipeline
6.2	diagram	Trade-journal database tables and links
6.3	graph	Funding rate history (real data if available)
6.4	graph	Open interest and price as two stacked panels
6.5	graph	Distribution of trade results in R
6.6	graph	Monte Carlo fan chart of equity with drawdown percentiles

Common mistakes. Mixing UTC and IST; survivorship in symbol choice; annualising crypto with 252 days; computing Sharpe on trades instead of on daily returns; a journal with no "mistake" field.

Hands-on exercise. Create the journal, load your testnet and minimum-size trades, run the review queries and a Monte Carlo on your backtest. Checklist of 8 items.

Verify before writing. Binance endpoints for candles, funding history and open-interest history (limits and how far back each goes); bulk data file layout.

Unit III · Automation

Chapter 7. The bridge: TradingView signal to Binance order

Why it matters. This is where a chart signal becomes a real order without you clicking. A small mistake here can repeat itself many times, so every step is validated.

Learning goals

- Explain webhooks and design a JSON alert message with a secret token.
- Build a FastAPI receiver with validation, HTTPS and duplicate protection.
- Translate a signal into a sized order with stop and target.
- Test locally, then end to end on the Binance testnet.

Sections

- **7.1 Webhooks from the basics; TradingView webhook requirements (paid plan**, 2FA, allowed ports).
- **7.2 Designing the alert message**: JSON with placeholders, a secret token, a unique signal ID.
- **7.3 The FastAPI receiver**: schema validation, constant-time token check, fast reply, queue, logging.
- **7.4 HTTPS**: a domain, a reverse proxy with automatic certificates, the firewall; never plain HTTP.

- **7.5 Signal-to-order translation**: side, quantity from risk (Chapter 3 calculator), rounding, leverage and margin checks, entry, stop and take-profit with reduce-only.
- **7.6 Testing**: unit tests, curl, dry-run mode, then testnet.

Code. `code/ch07/receiver/` (`app.py`, `schemas.py`, `translator.py`, `idempotency.py`), `tests/`, `Caddyfile.example`, a Pine `alert()` snippet, and a sample alert JSON.

Table O.8. Figures planned for Chapter 7.

Figure	Kind	Content
7.1	sequence	From Pine alert to Binance order to Telegram message
7.2	diagram	Deployment: domain, HTTPS proxy, FastAPI, firewall
7.3	flowchart	Validation and signal-to-order translation
7.4	diagram	Life of a signal as a state machine
7.5	graph	Time from alert to order in a testnet run (from your logs)

Common mistakes. Secret token in the URL; no duplicate protection, so one alert becomes two orders; plain HTTP; doing slow work before replying; trusting the price in the alert instead of the live price; missing stop if the entry fills but the stop order fails.

Hands-on exercise. Fire a test alert from TradingView to your receiver, see a testnet order with its stop and target, and receive a Telegram message. Checklist of 10 items.

Verify before writing. TradingView webhook rules (plans, 2FA, ports, timeout, content type, sending IP addresses) and alert placeholders; Binance order parameters for stop and take-profit.

Chapter 8. Engineering a reliable bot

Why it matters. A bot that works on a good day is easy. One that behaves safely when the internet drops, the exchange rejects an order, or prices gap is what protects your capital.

Learning goals

- Use the Binance USDS-M Futures API safely: keys, signing, timestamps, exchange rules.
- Respect rate limits and use WebSockets for live data and order updates.
- Build a risk engine with a daily loss limit, position caps and a kill switch.
- Reconcile local state with the exchange, log everything, send Telegram alerts, and run on a VPS.

Sections

- **8.1 The Futures API**: base addresses (live and testnet), signed requests, timestamps and receive window, official Python connector vs plain requests.
- **8.2 Keys and security**: a separate subaccount, no withdrawal rights, IP restriction, environment variables, key rotation.
- **8.3 Exchange rules**: price and quantity filters, minimum notional, position mode, conditional orders.
- **8.4 Rate limits**: request weight, order limits, handling 429 and 418 responses, backoff.
- **8.5 WebSockets**: market streams and the user-data stream (listen key, keep-alive, reconnects).
- **8.6 The risk engine**: risk per trade, max position, max leverage, daily loss limit (example: 3% = Rs 3,000), max trades per day, cool-down after losses, kill switch.
- **8.7 Reconciliation**: compare local and exchange state on a timer; fix orphan or missing stops.
- **8.8 Logging and Telegram alerts.**
- **8.9 VPS hosting**: choosing a region, systemd service, firewall, time sync, updates, backups.
- **8.10 Where Rust fits later**: what does not need it, and what might (fast stream processing, a small, very reliable executor), with a small example.

Code. `code/ch08/bot/` (config, REST client with signing, filters, risk, WebSocket user stream, reconcile, Telegram, kill switch, main loop), tests with mocked responses, `deploy/bot.service`, and a small Rust request-signing example (marked UNTESTED if Rust cannot be compiled in the writing environment).

Table O.9. Figures planned for Chapter 8.

Figure	Kind	Content
8.1	diagram	Bot modules and how they talk
8.2	graph	Request weight used per minute vs the limit (SIMULATED day)
8.3	sequence	User-data stream: create, keep alive, expire, reconnect
8.4	flowchart	Risk engine checks before every order, and the kill switch
8.5	diagram	Order state machine and the reconciliation loop
8.6	diagram	VPS deployment

Common mistakes. Keys with withdrawal rights; local clock drift; retrying a timed-out order without checking whether it went through; assuming the WebSocket never drops; risk limits that reset when the bot restarts; no alert when the bot itself stops.

Hands-on exercise. Run the bot on a VPS against testnet for 7 days; trigger the kill switch on purpose; break the network and watch reconciliation repair state. Checklist of 10 items.

Verify before writing. Binance base addresses, signing methods, rate limits, listen-key rules, conditional (algo) order endpoints, client order IDs; Telegram Bot API; server location advice.

Chapter 9. Launching an AI agent

Why it matters. Agents can save you hours of research and watch your positions while you are in class. They can also make confident mistakes. This chapter builds agents that report and propose, while you keep the final say.

Learning goals

- Describe an agent as model + tools + memory + loop.
- Choose between research, monitoring and execution agents and give each the right permissions.
- Combine both MCP servers in Claude, schedule tasks, and build an agent with the Claude Agent SDK.
- Add guardrails: human approval, hard limits outside the model, and defences against prompt injection.

Sections

- **9.1 What an agent is**, from the basics.
- **9.2 Three roles**: research (read-only, writes reports), monitoring (reads positions, sends alerts), execution (proposes orders; a human approves).
- **9.3 Claude with both MCP servers**: from analysis to an order proposal with the safety checklist.
- **9.4 Scheduled tasks**: running Claude on a timetable, and plain cron with a headless run.
- **9.5 The Claude Agent SDK in Python**: tools, permission settings, hooks that block write tools.
- **9.6 Binance Skill Hub**: what skills are, how to install them, and how to vet third-party skills.
- **9.7 Guardrails**: allow-lists, read-only by default, human approval, limits enforced by the exchange and the risk engine, logs of every tool call, a kill switch.
- **9.8 Risks**: invented numbers, overtrading, runaway loops, prompt injection from news or web pages, running costs.

Code. `code/ch09/research_agent.py`, `monitor_agent.py`, `approval_gate.py` (a hook that blocks write tools unless you approve), scheduling examples. Offline parts are tested; parts that need a paid API key are marked UNTESTED unless one is available.

Table O.10. Figures planned for Chapter 9.

Figure	Kind	Content
9.1	diagram	The agent loop: model, tools, memory
9.2	diagram	Three agent roles and their permissions
9.3	diagram	Layers of guardrails (defence in depth)
9.4	sequence	An execution proposal with human approval
9.5	graph	Running cost vs how often the agent checks (from published API prices)

Common mistakes. Giving a research agent order tools; approving proposals without the checklist; letting an agent read untrusted web text and then act; no budget cap on API usage; no log of what the agent did.

Hands-on exercise. Run a daily research agent and a monitoring agent for one week on testnet positions; review every log entry. Checklist of 9 items.

Verify before writing. Claude Agent SDK documentation (package name, options, hooks, permissions); Claude Code scheduling options; Binance Skill Hub contents and install steps; Claude API pricing.

Unit IV · Growth and the road ahead

Chapter 10. The new technology landscape

Why it matters. New tools arrive every week, each claiming an edge. This chapter gives you a calm way to judge them.

Learning goals

- Describe on-chain analytics, derivatives data and sentiment tools, and what each can tell a futures trader.
- Explain on-chain perpetual exchanges and how their risks differ from Binance.
- Understand agent frameworks and agent payments (x402).
- Test a popular claim with data and separate real edge from hype.

Sections

- **10.1 On-chain analytics**: exchange flows, stablecoin supply, large-wallet tracking.
- **10.2 Derivatives data**: funding, open interest, liquidations, basis.
- **10.3 Sentiment tools and their traps.**
- **10.4 On-chain perpetual exchanges**: self-custody, order books on a blockchain, oracle and mark price design, smart-contract risk, and the legal and tax position for Indian users.
- **10.5 Agent frameworks and the MCP ecosystem.**
- **10.6 Agent payments**: x402, Binance x402 and agent wallets.
- **10.7 Edge vs hype**: a five-question test (Is the data unique? Is it early? Can it be tested? Does it survive costs? Will it last when others use it?).
- **10.8 What stronger AI realistically changes**: faster research and better code for everyone, so simple edges fade faster; risk does not go away.

Code. `code/ch10/funding_extremes_test.py` (does extreme funding predict the next day's return?) and `decay_sim.py` (how an edge fades as more traders use it).

Table O.11. Figures planned for Chapter 10.

Figure	Kind	Content
10.1	diagram	Landscape map: data, analysis, execution, payments; exchanges vs on-chain
10.2	flowchart	The edge-vs-hype test
10.3	graph	Next-day return by funding-rate group (real data if available)
10.4	graph	How an edge decays as adoption grows (SIMULATED)
10.5	diagram	Exchange perpetual vs on-chain perpetual: where the risks sit

Common mistakes. Paying for data before testing whether it helps; trusting dashboards without knowing how numbers are built; moving funds on-chain without understanding custody and bridge risk; treating legal grey areas as safe.

Hands-on exercise. Pick one tool or claim you have seen online and run it through the five-question test with data. Checklist of 7 items.

Verify before writing. Current status, pricing and availability in India of every product named; the x402 specification; FIU-IND rules for offshore platforms.

Chapter 11. Growing Rs 1,00,000 responsibly

Why it matters. How you grow capital decides whether a small edge survives. This chapter replaces "how fast can I grow?" with "how do I avoid ruin while I find out whether I have an edge?"

Learning goals

- Use a staged capital ladder with clear rules for moving up and down.
- Compute realistic compounding and expected drawdowns.
- Understand Indian tax on crypto (VDA rules, TDS, reporting) and keep proper records.
- Write and follow a one-page trading plan with psychology rules.

Sections

- **11.1 The staged capital ladder**: testnet; Rs 10,000 at minimum size; Rs 25,000; Rs 50,000; Rs 1,00,000. Promotion only after enough trades with rule-following and results inside the backtest's range; automatic step-down after a set drawdown.
- **11.2 Realistic compounding**: what 1%, 2% and 3% a month look like after costs and tax, and the drawdowns that come with them; why most leveraged retail accounts lose money.
- **11.3 Indian tax on crypto**: the 30% VDA tax plus cess, no deductions except cost of acquisition, no loss set-off or carry-forward, 1% TDS, Schedule VDA in the return, the new Income-tax Act 2025 section numbers, and the unsettled treatment of crypto futures. GST on exchange fees. When to see a chartered accountant.
- **11.4 Record-keeping**: exports, funding and fee records, transfers, rupee on-ramp records, TDS statements; how long to keep them.
- **11.5 Psychology rules**: pre-commitment, maximum trades per day, stop after two losses, no revenge trading, sleep and exam weeks.
- **11.6 Your written trading plan** (template).

Code. `code/ch11/ladder_sim.py`, `vda_tax_estimator.py` (education only, assumptions printed), `trading_plan_template.md`.

Table O.12. Figures planned for Chapter 11.

Figure	Kind	Content
11.1	diagram	The capital ladder with promotion and step-down rules
11.2	graph	12-month equity fan at 0.5% and 1% risk, and a no-edge strategy after costs (SIMULATED)
11.3	graph	Range of yearly max drawdowns (SIMULATED)
11.4	graph	Tax with no loss set-off vs with netting, on the same 10 trades
11.5	diagram	Record-keeping flow from exchange to tax return

Common mistakes. Adding money after losses; moving up the ladder after a lucky month; assuming losses reduce tax; not keeping rupee on-ramp records; relying on the exchange's tax report alone.

Hands-on exercise. Write your trading plan, set your ladder rules, and set up your records folder. Checklist of 8 items.

Verify before writing. Income Tax Department and CBDT material for tax year 2026-27 (VDA sections under the Income-tax Act 2025, TDS rules and thresholds, ITR schedule), any Finance Act 2026 changes, GST on exchange fees. All tax content will carry dates and a "consult a CA" note.

Chapter 12. Capstone and career

Why it matters. The system is only useful if you can measure it honestly and explain it well. This chapter ties everything together and turns your work into evidence for a career in research and fund management.

Learning goals

- Run the full system with a daily, weekly and monthly routine.
- Compare 90 days of live results with the backtest and decide: continue, reduce or stop.
- Build a portfolio that shows skill and honesty to employers in quant, research and fund roles.

Sections

- **12.1 The full system running together**, and the runbook.
- **12.2 The 90-day live review**: trade count, win rate, average R, slippage, fees, funding and drawdown vs the backtest; is live performance inside the Monte Carlo range?
- **12.3 The decision**: continue, reduce size, or stop and research.
- **12.4 Building a portfolio**: repository structure, research write-ups, honest performance reporting, how this work maps to the CFA curriculum (quantitative methods, derivatives, portfolio management, ethics), roles in India (AMCs, PMS, AIFs, research, exchanges), and relevant certifications.

Code. `code/ch12/live_vs_backtest.py` (reads the journal database and backtest trades, writes a report) and `capstone_report_template.md`.

Table O.13. Figures planned for Chapter 12.

Figure	Kind	Content
12.1	diagram	The full system with the daily, weekly and monthly runbook
12.2	graph	Live equity vs the backtest's Monte Carlo band (SIMULATED until your 90 days are done)
12.3	graph	Slippage and fees: backtest assumption vs live distribution
12.4	flowchart	Continue, reduce or stop
12.5	diagram	Career map: roles, skills and the evidence for each

Common mistakes. Changing the strategy during the review period; comparing live results with the best backtest instead of the out-of-sample one; showing only winning trades in a portfolio; claiming returns without drawdowns.

Hands-on exercise. Produce your capstone report and publish a clean portfolio repository. Checklist of 10 items.

Verify before writing. Current CFA curriculum topic areas; NISM certification names; any SEBI rules relevant to your planned roles.

After Chapter 12: the combined book

- One combined Word file: cover page, table of contents, preface, all 12 chapters, glossary, and an index of prompts (every prompt box, with its chapter and page).
- A PDF export of the same file.
- `PROGRESS.md` updated with every fact that could not be verified.